

Stress-Testing Semantic Robustness in Vision–Language–Action Policies

An Empirical Study of Instruction–Surface Sensitivity in Octo

Seoyoon Yum

KAIST

gilbertyum12960@gmail.com

June 20, 2026

Abstract

Vision–Language–Action (VLA) policies are trained to ground natural-language instructions into embodied behavior, and their value rests on an implicit assumption: that a policy should respond identically to instructions that mean the same thing but are phrased differently. We stress-test this assumption with a controlled, single-task probe. Holding the policy weights and the visual scene fixed, we evaluate the Octo-Small generalist policy on the `SIMPLER widowx_put_eggplant_in_basket` task under a baseline instruction and ten families of programmatically generated, meaning-preserving instruction mutations, for 11 conditions $\times$ 20 seeds = 220 episodes. Success rate drops from 60% on the baseline instruction to a 25% mean across the perturbation families (a $\sim$ 58% relative decline; 28.2% pooled across all conditions). The robust pattern is a split between *structural* and *additive* edits: the one edit that genuinely reorders the sentence—fronting the destination phrase (“Into yellow basket, put eggplant”)—collapses to 0%, and substituting the core action verb or preposition for a near-synonym is severe (10–15%), whereas *additive* edits such as added descriptors, directional phrases, and verbosity are comparatively tolerated (40–45%). We read this as evidence of template-matching on canonical word order rather than abstract semantic grounding. We do *not* draw conclusions about passive voice or negation: as implemented, those two operators are templated insertions rather than true grammatical transformations, so their scores are reported but not interpreted as evidence about those phenomena. We report Wilson confidence intervals throughout; with $n = 20$ they are wide and overlapping, so the findings should be read as effect bands rather than precise rankings. This is an exploratory study on a single policy and task and is not intended as a benchmark; code and configurations are released for reproduction and extension.

1 Introduction

Vision–Language–Action (VLA) models aim to map a natural-language instruction and a visual observation onto low-level robot actions, and recent generalist policies have made this mapping increasingly broad [1, 2, 3, 5]. A central premise behind such systems is *linguistic invariance*: two instructions that are semantically equivalent but lexically or syntactically different should elicit the same behavior. Human users, however, almost never express a command in a single canonical form. The same intent arrives as a terse imperative, a polite request, a passive construction, or a sentence with the destination stated first. If a policy is robust only to the phrasing it happened to see during training, its competence is narrower than its success rate on a canonical test set suggests.

This gap—between what a policy appears to *understand* linguistically and what it actually *executes*—is what we call the semantic–behavioral gap, and it is safety-relevant: in high-stakes

settings, sensitivity to surface form is sensitivity to who happens to be giving the order. This report asks a narrow, empirical question:

How do semantically equivalent variations in an instruction affect the success rate (SR) of a VLA policy?

Hypothesis. Meaning-preserving mutations will expose sensitivity to linguistic *surface form*—word order, voice, lexical choice—rather than to the abstract task semantics, indicating that the policy relies on template-matching against canonical instructions.

Contributions. This is a small, deliberately scoped study. Its contributions are: (i) a config-driven harness that injects controlled instruction mutations into a fixed simulated rollout; (ii) a taxonomy of ten meaning-preserving mutation families with representative transformations of a single canonical instruction; (iii) a per-condition success-rate breakdown for the Octo policy on one SIMPLER task, reported with Wilson confidence intervals; and (iv) a failure-mode analysis distinguishing structural from additive edits. We state the limitations of this design (Section 7) prominently rather than as an afterthought.

2 Background and Related Work

Generalist robot policies. Large policies pretrained on diverse robot data—RT-2 [1], Octo [2], OpenVLA [3], and flow-based successors such as π_0 [6], trained on aggregations like Open X-Embodiment [5]—are explicitly designed to follow free-form language. Octo [2] is a transformer-based policy pretrained on 800k trajectories that can be conditioned on language commands or goal images; we use its WidowX/Bridge configuration here.

Simulated evaluation. Real-robot evaluation is expensive and hard to reproduce. SIMPLER [4] provides simulated environments for common real robot setups (Google Robot, WidowX+Bridge) that correlate with real-world performance and reflect policy behavior modes such as sensitivity to distribution shift. SIMPLER’s stated focus is largely *visual* distribution shift (backgrounds, lighting, distractors, textures); the present study repurposes the same controlled setting to probe *linguistic* distribution shift on the instruction channel, which is comparatively underexamined for VLA policies.

3 Method

3.1 Environment and policy

We use the SIMPLER simulator [4] with the **Octo-Small** policy [2] in its `widowx_bridge` configuration, on the task `widowx_put_eggplant_in_basket`, viewed through the third-person camera. Policy weights and the visual scene are held fixed across all conditions; the only manipulated variable is the text of the instruction. Each episode runs for up to 120 timesteps of closed-loop control: at every step the policy is queried with the current RGB observation and the (mutated) instruction, and its predicted translation and rotation are multiplied by a fixed action scale before being sent to the environment. To stabilize grasping we apply a sticky-gripper heuristic: once the policy first commands a close, the gripper is held closed for the next 10 steps. All conditions use the same fixed set of seeds 0–19 (20 per condition); the seed determines both the environment reset and—for mutated conditions—the mutation generator’s randomness, making each rollout deterministic.

Table 1: Instruction-mutation families with *actual* generator outputs for the baseline instruction “*put eggplant into yellow basket.*” Where an operator produced several seed-dependent forms, one or two are shown.

Mutation family	Actual generated output
Synonym substitution	“place eggplant inside yellow basket” / “position eggplant in yellow basket”
Passive voice	“put eggplant into yellow basket should be done.”
Spatial reordering	“Into yellow basket, put eggplant”
Formal / informal	“deposit eggplant into yellow basket” / “Hey, put eggplant into yellow basket”
Verb rephrasing	“set down eggplant into yellow basket” / “drop off eggplant into yellow basket”
Object descriptors	“put purple eggplant into yellow basket” / “put fresh eggplant into yellow basket”
Directional language	“put eggplant into yellow basket toward the target.”
Temporal modifiers	“put eggplant into yellow basket right away.”
Negation / positive	“Don’t forget to put eggplant into yellow basket” / “Make sure to put eggplant into yellow basket”
Complexity variation	“Carefully put eggplant into yellow basket”

Action-scale calibration and baseline gate. The action scale is not assumed but calibrated: before the main run we sweep scales $\{0.25, 0.5, 0.75, 1.0\}$ over 5 baseline episodes each and lock in the value with the highest success rate, which here is 0.75. We then run the 20 baseline trials and only proceed to the mutation conditions if the baseline success rate clears a preset floor (set to 0.30 in this run). The locked run recorded a baseline rate of 0.60, above the floor, so the mutation conditions proceeded.

3.2 Instruction-mutation taxonomy

The baseline is the environment-provided instruction for the task, “*put eggplant into yellow basket.*”. Mutations are not hand-written but *programmatically generated* by a rule-based mutation module: for each family, a deterministic operator applies regular-expression substitutions (swapping verbs, prepositions, or descriptors) or a templated rewrite, seeded by the trial seed. Because the seed drives the operator’s random choices, a single family can yield from one to several distinct surface strings across the 20 seeds (e.g. synonym substitution produced eight distinct forms, while temporal and complexity edits produced one). Table 1 shows actual operator outputs for this task. The operators are intentionally lightweight, and two of them approximate their linguistic category rather than realizing it fully: the “passive voice” operator appends a nominalized clause (“... *should be done.*”) rather than performing a full passive transformation, and “negation/positive” prepends an emphatic lead (“*Don’t forget to.*”) rather than constructing the task through logical exclusion. The family labels should therefore be read as the *intended* transformation. All outputs preserve the task goal (placing the eggplant in the basket).

3.3 Protocol

Each condition (the baseline plus the ten mutation families) is evaluated over the same 20 seeds, giving $11 \times 20 = 220$ episodes in total. The runner is config-driven: a single YAML file specifies the seeds, task, camera, step budget, action-scale sweep, and baseline floor. For each (family, seed) pair

Table 2: Success rate by condition over 20 trials each, with 95% Wilson confidence intervals. The qualitative band is a coarse descriptor, not a statistical claim; the confidence intervals overlap substantially.

Condition	Succ./Trials	SR	95% CI	Band
Baseline	12/20	60%	[39%, 78%]	reference
Directional language	9/20	45%	[26%, 66%]	relatively robust
Complexity variation	9/20	45%	[26%, 66%]	relatively robust
Object descriptors	8/20	40%	[22%, 61%]	moderate
Formal / informal	8/20	40%	[22%, 61%]	moderate
Negation / positive	6/20	30%	[15%, 52%]	sizable drop
Temporal modifiers	4/20	20%	[8%, 42%]	noticeable drop
Synonym substitution	3/20	15%	[5%, 36%]	large drop
Verb rephrasing	2/20	10%	[3%, 30%]	near-failure
Passive voice	1/20	5%	[1%, 24%]	near-failure
Spatial reordering	0/20	0%	[0%, 16%]	total failure
Mutation mean	50/200	25%	—	—
Pooled (all)	62/220	28.2%	—	—

the mutation is regenerated from the environment instruction with the seed, and a structured CSV logs the per-episode outcome—success, episode length, the original and mutated instruction strings, and (where the environment exposes it) a distance-to-target value. The primary metric is binary success rate (SR) per condition. In this environment the distance-to-target field was not populated, so the analysis is binary-only (Section 7).

3.4 Uncertainty quantification

Because each condition has only $n = 20$ binary trials, point estimates of SR are noisy. We accompany every SR with a 95% Wilson score interval, which behaves well for small n and near-boundary proportions (e.g., 0/20 and 1/20):

$$\text{CI}_{95} = \frac{\hat{p} + \frac{z^2}{2n}}{1 + \frac{z^2}{n}} \pm \frac{z}{1 + \frac{z^2}{n}} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n} + \frac{z^2}{4n^2}}, \quad z = 1.96.$$

4 Results

Table 2 and Figure 1 report SR by condition. The baseline instruction succeeds in 12/20 trials (60%). Across the ten mutation families the mean SR is 25% (50/200), a relative decline of roughly 58% from baseline; pooled over all 11 conditions the rate is 62/220 $\approx$ 28.2%.

5 Failure-Mode Analysis

Catastrophic: genuine reordering. The clean, interpretable failure is *spatial reordering*: fronting the destination phrase (“Into yellow basket, put eggplant”) while leaving every word otherwise intact drives SR to 0/20. Because this is the one operator that truly disturbs word order without changing the lexicon, it is the strongest single piece of evidence that the policy is tied to a canonical verb-first imperative and breaks outright when that order is disturbed, rather than degrading gracefully. The *passive voice* condition is also near-zero (1/20), but we read it cautiously: as implemented it

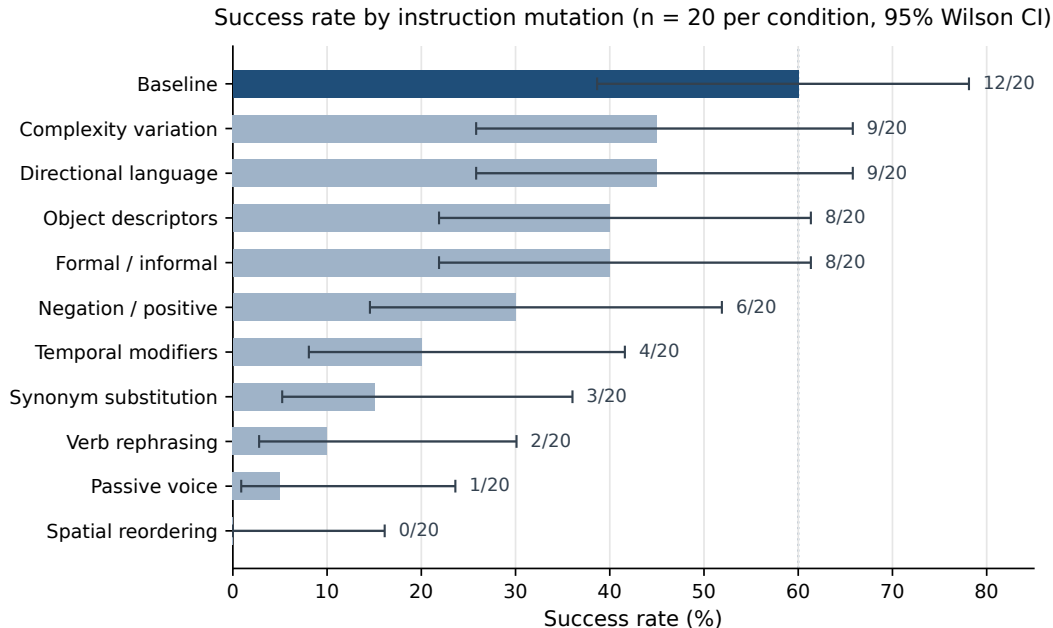


Figure 1: Success rate by condition (baseline highlighted), sorted by SR. Whiskers are 95% Wilson intervals; the dotted line marks the baseline rate. With $n = 20$ per condition the intervals are wide and heavily overlapping—differences of a few trials (e.g. 40% vs. 45%) are not statistically separable. Note that the “passive voice” and “negation/positive” operators are templated approximations rather than true grammatical transformations (Section 7); the interpretable signal is the split between structural and additive edits, not the per-label ranking.

appends a nominalized clause (“... should be done.”) rather than producing a true passive, so it conflates word-order disruption with a trailing, partly degenerate construction and should not be taken as evidence about passive voice as such.

Severe: core-token substitution. *Verb rephrasing* (2/20) and *synonym substitution* (3/20) are the next worst. Here the operators swap the load-bearing verb and preposition for near-synonyms while leaving word order intact—“put”→“set down” or “drop off” (e.g. “set down eggplant into yellow basket”), and “put”→“place”/“position” with “into”→“in”/ “inside” (e.g. “place eggplant inside yellow basket”). That such minimal lexical changes collapse success points to anchoring on the exact training tokens “put”/“into” rather than to a learned synonymy.

Relative robustness pockets. Directional language and complexity variation (45% each), object descriptors and formal/informal tone (40% each) retain a substantial fraction of baseline performance. These edits are *additive or structure-preserving*: they prepend context, add adjectives, or change register while leaving the canonical verb–object–destination skeleton intact. That added descriptive “noise” does not consistently hurt suggests the policy filters some non-essential tokens.

Summary observation. Across conditions, the operative axis is not how *complex* an instruction is but whether it *preserves canonical structure*. Additive changes are tolerated; structural re-orderings are not. Robustness does not decline monotonically with linguistic complexity.

6 Discussion

The pattern is consistent with the hypothesis: failure under meaning-preserving reordering implies surface-form template-matching rather than abstract semantic grounding. A policy that had genuinely grounded “put X in Y” should be largely indifferent to whether Y is named before or after X; the observed total collapse under reordering indicates otherwise. Practically, structural sensitivity narrows real-world deployability: non-expert users phrase commands freely, and a policy that only reliably executes the canonical template will appear far less capable in the field than its canonical-instruction SR implies.

These conclusions are bounded by uncertainty. With $n = 20$, the Wilson intervals in Table 2 are wide (typically ± 18 –20 percentage points) and overlap heavily. The fine-grained ordering within a band—40% vs. 45%, for instance—is not statistically meaningful here; the robust finding is the *gap between the structural-failure cluster and the additive-tolerance cluster*, not the rank order within each.

7 Limitations

This study is exploratory and its scope is deliberately narrow.

- **Single policy, single task.** All results are from Octo-Small on one SIMPLER WidowX task. We cannot separate “Octo-specific” brittleness from a property of VLA policies in general; multi-policy and multi-task validation is required before any general claim.
- **Weak baseline.** The baseline SR is 60%, well below a robust operating point—and below the project’s own pre-registered $\geq 90\%$ baseline threshold, which was not met. The policy is already near its competence ceiling on this task, so some of the observed drops may reflect a fragile baseline rather than a pure language effect.
- **Binary metric only.** We record success/failure with no partial-completion signal (e.g. distance-to-target, grasp-without-place), which would distinguish “misunderstood the instruction” from “understood but failed to execute.” The harness attempts to log distance-to-target, but the field was not populated by this environment.
- **Rule-based, label-approximate mutations.** Mutations are generated by lightweight deterministic operators, not a human-authored or human-validated set, and the seed yields only a small number of surface forms per family (one to eight). Two operators approximate their category rather than realizing it—the “passive voice” and “negation/positive” families are templated insertions, not true grammatical transformations—so per-family difficulty partly reflects these authoring choices. Semantic equivalence was not independently verified.
- **Calibration coupling.** The action scale was tuned on the baseline instruction, which may marginally favor the baseline condition relative to the mutated ones.
- **Small samples.** $n = 20$ per condition yields wide confidence intervals; the design supports band-level conclusions, not precise effect sizes.
- **No interventions.** We do not test mitigations (instruction fine-tuning, paraphrase augmentation, prompt normalization), so the results characterize the gap without addressing it.

8 Conclusion

On a single VLA policy and task, perturbing the phrasing of an instruction while preserving its goal reduced success rate from 60% (baseline) to a 25% mean across ten perturbation families (28.2% pooled). The interpretable signal is a split between structural and additive edits: the one

operator that genuinely reorders the sentence (fronting the destination phrase) collapsed to 0%, and substituting the core action verb or preposition was severe (10–15%), whereas additive edits—added descriptors, directional phrases, register, verbosity—were comparatively tolerated (40–45%). This is consistent with template-matching on canonical word order rather than semantic grounding. We deliberately do not claim results about passive voice or negation, whose operators are templated approximations here. We emphasize that this is an exploratory probe, not a benchmark: the small samples and single-policy scope mean the contribution is a documented effect and a reusable harness, not a general law.

Future work. The natural extensions are (i) multiple policies (e.g. OpenVLA [3], π_0 [6]) to test generality, which the config-driven harness makes a cheap swap—the released code already includes an OpenVLA inference entry point; (ii) multiple tasks and stronger baselines; (iii) partial-credit metrics; (iv) automatically generated, human-validated paraphrase sets; and (v) robustness-aware training objectives that explicitly enforce invariance to meaning-preserving rewrites.

Reproducibility. The reported numbers correspond to the locked single-task run (Octo-Small, action scale 0.75, 120 steps, sticky-gripper window 10, seeds 0–19) configured by `octo_experiment_config.yaml` and recorded as the final block of `data/results/results_octo.csv`; that committed file also accumulates earlier development runs, so statistics computed over the whole file will differ from the per-condition run reported here. Code, configs, the mutation generator, and per-episode logs are available at <https://github.com/SeoyoonYum/VLA-Semantic-Robustness-Test>.

References

- [1] A. Brohan, N. Brown, J. Carbajal, et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. *arXiv preprint arXiv:2307.15818*, 2023.
- [2] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, et al. Octo: An Open-Source Generalist Robot Policy. In *Robotics: Science and Systems (RSS)*, 2024. arXiv:2405.12213.
- [3] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, et al. OpenVLA: An Open-Source Vision-Language-Action Model. *arXiv preprint arXiv:2406.09246*, 2024.
- [4] X. Li, K. Hsu, J. Gu, K. Pertsch, O. Mees, H. R. Walke, et al. Evaluating Real-World Robot Manipulation Policies in Simulation. In *Conference on Robot Learning (CoRL)*, 2024. arXiv:2405.05941.
- [5] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024. arXiv:2310.08864.
- [6] K. Black, N. Brown, D. Driess, et al. π_0 : A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164*, 2024.